# Лекция 13. Гибридные методы управления мультиагентными робототехническими системами

*Гибридные автоматы, иерархии, нечёткая логика, переключения, события и барьерные функции*

90 минут · две части по 45

Курс «Управление распределёнными робототехническими системами»

# План лекции

1. Зачем нужен гибрид: почему одного «чистого» метода группе роботов мало
2. Гибридный автомат: режимы, инварианты, охранные условия и сбросы
3. Как поделить решения между центром и роботами: три схемы координации
4. Трёхуровневая иерархия управления и разделение темпов между уровнями
5. Комбинирование методов: рой с обучением, теория игр с консенсусом
6. Нечёткая логика: как словесные правила эксперта становятся управлением
7. Нейро-нечёткие системы: обучаемая нечёткая сеть ANFIS
8. Переключаемые системы: когда переключения ломают устойчивость
9. Событийное управление: передавать данные только по необходимости
10. Барьерные функции безопасности (CBF, Control Barrier Functions) как фильтр команд
11. Сборка и проверка гибридного контроллера робота

*Синие — первая часть · янтарные — вторая часть*

# Что нужно помнить с прошлых занятий

*Сегодня мы постоянно будем опираться на эти шесть идей из прошлых лекций — освежим их до начала новой темы.*

| Понятие (лекция N) | Что это простыми словами | Где пригодится сегодня |
|---|---|---|
| Централизованная и децентрализованная архитектура (лекция 3) | Решает один центр — или каждый робот сам, по данным от соседей | Гибридная архитектура сочетает обе на разных уровнях |
| Протокол консенсуса (лекция 4) | Робот подтягивает своё значение к соседским, и группа сходится к общему | Консенсус держит строй и согласует цены в гибридных схемах |
| Функция Ляпунова (лекция 4) | Аналог энергии: не меньше нуля и убывает при движении к равновесию | Через неё доказывают устойчивость при переключениях режимов |
| Конечный автомат (лекция 8) | Набор состояний робота и переходов между ними по событиям | Это дискретная часть гибридного автомата и супервизор режимов |
| Проверка модели, model checking (лекция 8) | Программа перебирает все состояния автомата в поиске ошибки | Так проверяют супервизор: нет тупиков, авария достижима |
| Правила Рейнольдса и CTDE (лекции 5 и 7) | Три правила стаи; обучение с общим критиком и локальное исполнение | Обучение с подкреплением настраивает роевые правила |

# Сокращения и термины лекции

*Все сокращения первой части лекции собраны здесь; к таблице можно вернуться в PDF в любой момент.*

| Обозначение | Расшифровка | Смысл в одной фразе |
|---|---|---|
| ПИД | Пропорционально-интегрально-дифференциальный регулятор | Быстрый регулятор привода по ошибке, её интегралу и производной |
| MILP | Mixed-Integer Linear Programming — целочисленное программирование | Оптимизация с целыми переменными: кому какое задание назначить |
| MPC | Model Predictive Control — управление с прогнозирующей моделью | На каждом такте оптимизирует движение на несколько шагов вперёд |
| LQR | Linear-Quadratic Regulator — линейно-квадратичный регулятор | Оптимальная обратная связь для линейной модели объекта |
| RL | Reinforcement Learning — обучение с подкреплением | Агент учится действовать по сигналу награды, пробуя и ошибаясь |
| CTDE | Centralized Training, Decentralized Execution | Обучение централизованное, а исполнение — каждым роботом по своим данным |
| STRIPS | Stanford Research Institute Problem Solver | Классический символический планировщик: план как цепочка действий |

# Сокращения и термины лекции (продолжение)

*Сокращения второй части лекции: нечёткие сети, барьерные функции и инструменты проверки.*

| Обозначение | Расшифровка | Смысл в одной фразе |
|---|---|---|
| PDDL | Planning Domain Definition Language — язык описания задач планирования | Стандартный язык, на котором планировщику описывают действия и цели |
| ЛМН | Линейные матричные неравенства | Условия на матрицы, которые компьютер проверяет численно |
| ANFIS | Adaptive Neuro-Fuzzy Inference System — нейро-нечёткая система вывода | Нечёткий регулятор в виде сети, параметры которой учатся по данным |
| CBF | Control Barrier Function — барьерная функция управления | По ней команду поправляют так, чтобы робот не покинул безопасную зону |
| QP | Quadratic Program — задача квадратичного программирования | Минимум квадратичной функции при линейных ограничениях; решается быстро |
| ИНС | Инерциальная навигационная система | Оценивает положение робота по гироскопам и акселерометрам |
| UPPAAL, SPIN | Названия программ автоматической проверки моделей | UPPAAL проверяет временные автоматы, SPIN — дискретные модели |

# Сокращения и термины лекции: основные понятия

*Эти понятия встречаются почти на каждом слайде; здесь они объяснены словами и на примере склада.*

| Термин | Что это значит | Пример со склада |
|---|---|---|
| Гибридная система | Непрерывное движение сочетается с дискретной сменой режимов | Робот едет плавно, но переключается: «к стеллажу», «объезд», «стоп» |
| Режим и инвариант | Режим — дискретное состояние; инвариант — где в нём можно оставаться | Режим «к стеллажу» допустим, пока препятствие дальше порога |
| Охранное условие и сброс | Условие, разрешающее переход; сброс — пересчёт состояния при переходе | Препятствие ближе порога: переход в «объезд», таймер режима обнулён |
| Устойчивость | Малые отклонения не растут и со временем затухают | Сбитый с маршрута робот сам возвращается на маршрут |
| Функция Ляпунова | Аналог энергии отклонения: не меньше нуля и всё время убывает | Квадрат отклонения от маршрута уменьшается с каждым тактом |
| Время пребывания (dwell time) | Минимальное время, которое система обязана провести в режиме | Супервизор не меняет режим, пока не прошло $\tau_d$ секунд |
| Барьерная функция | Функция h(x), которая не меньше нуля, пока система безопасна | Квадрат расстояния между роботами минус квадрат допуска |

# Часть 1. Гибридная динамика и архитектуры координации

*Мы определим гибридную систему, построим её модель — гибридный автомат — и разберём, как делят управление между центром, уровнями иерархии и разными методами.*

# 1 Почему одного «чистого» метода не хватает

*Начнём с вопроса, ради которого существует гибридное управление: почему группе роботов мало одного алгоритма.*

– Задачи группы тянутся от распределения заданий между роботами до стабилизации моторов колёс, и это разные масштабы времени.

– Планировщик хорошо решает стратегические задачи, но не успевает за жёстким реальным временем в миллисекунды.

– ПИД-регулятор (пропорционально-интегрально-дифференциальный) держит быстрый контур, но не принимает стратегических решений.

– Обучаемые модули приспосабливаются к новой среде, а аналитические регуляторы дают гарантии; нужны и те и другие.

– Поэтому реальные системы — группы беспилотников, складские флоты, беспилотный транспорт — собраны из нескольких методов.

**Пример со склада**

Сервер раздаёт роботам задания и резервирует коридоры, супервизор каждого робота выбирает режим поведения, а регуляторы колёс работают сотни раз в секунду. В одной системе работают как минимум три разных метода.

# 1 Два смысла термина «гибридный»

*Слово «гибридный» в лекции звучит в двух смыслах — разведём их сразу, чтобы потом не путаться.*

– В формальном смысле гибридная система сочетает непрерывную динамику $\dot{x} = f(x, u, t)$ и дискретную смену режимов.

– Дискретная часть — конечное множество режимов $Q = \{q_1, \ldots, q_m\}$ и правила переключения между ними по логическим условиям.

– В инженерном смысле гибридным называют метод, который комбинирует разнородные подходы: аналитические и обучаемые, центр и рой.

– Смыслы связаны: набор регуляторов с логикой переключения между ними формально и есть гибридная система.

– На складе колёса робота движутся непрерывно, а поведение делится на режимы «к стеллажу», «объезд», «ожидание», «стоп».

**Определение**

Гибридная система — динамическая система, в которой взаимодействуют непрерывная часть (дифференциальные уравнения) и дискретная (режимы и переключения). Здесь x — состояние робота (координаты, скорости), u — управление, t — время.

“

Практически любой автономный робот и любая мультиагентная система являются гибридными — вопрос лишь в том, признали ли мы это при проектировании.

*Главный тезис главы 13 пособия*

# 2 Гибридный автомат: режимы и непрерывное движение

*Чтобы работать с гибридностью строго, нужна модель. Стандартная модель — гибридный автомат Р. Алура и Т. Хенцингера.*

– Гибридный автомат задаётся восьмёркой H = (Q, X, f, Init, Inv, E, G, R); разберём её как ответы на четыре вопроса.

– Где я нахожусь: Q — конечное множество режимов, а $X \subseteq \mathbb{R}^n$ — непрерывное пространство состояний робота.

– Как я двигаюсь: в режиме q состояние меняется по своему закону $\dot{x} = f_q(x)$, и у каждого режима закон свой.

– Где можно оставаться: инвариант Inv(q) — множество состояний, при которых системе разрешено находиться в режиме q.

– С чего начинаю: Init — множество допустимых начальных пар «режим и состояние».

**Пример**

Термостат: режимы — «нагрев» и «охлаждение», непрерывное состояние — температура. Инвариант режима «нагрев» — «температура ниже верхнего порога»: пока это верно, нагреватель может оставаться включённым.

# 2 Гибридный автомат: переходы, охранные условия и сбросы

*Режимы и законы движения описаны; осталось сказать, когда и как автомат прыгает из одного режима в другой.*

– Куда можно прыгнуть: E — множество разрешённых переходов между режимами, то есть дуг графа режимов.

– Охранное условие G(e) (guard) открывает переход e: он разрешён, только когда состояние попало в множество G(e).

– Сброс R(e, x) (reset) задаёт новое состояние сразу после перехода, например скачок скорости шагающего робота при ударе стопы.

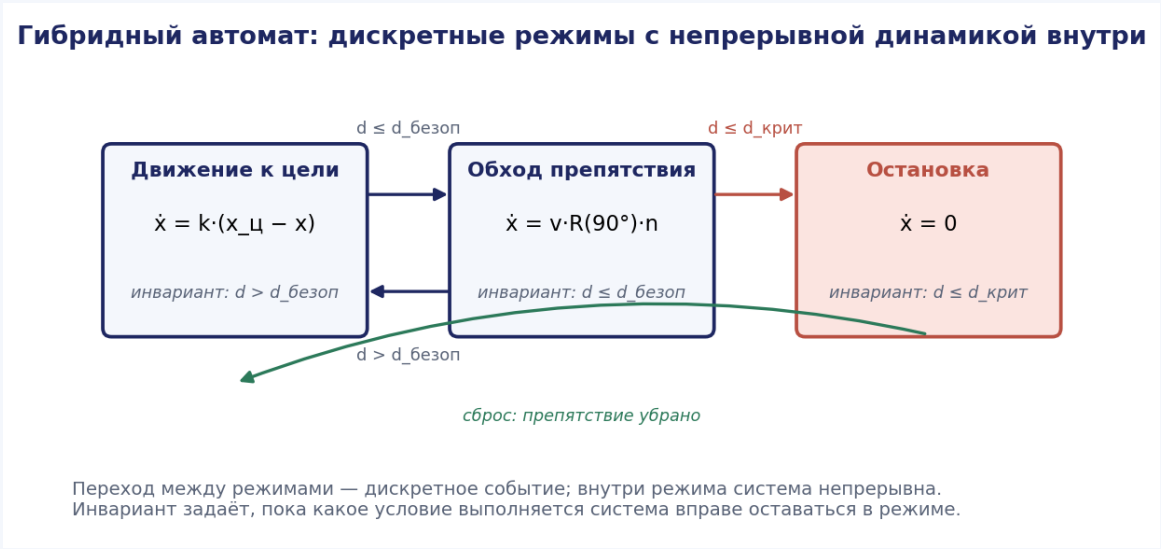
– Траектория чередует «течения» (flow) внутри инвариантов и мгновенные «скачки» (jumps) при срабатывании охранных условий.

**Пример со склада**

Охранное условие «расстояние до препятствия d < d_безоп» переводит робота из режима «к стеллажу» в режим «объезд». Условие «‖x − x_цель‖ < ε» означает, что робот подъехал к стеллажу ближе допуска ε, и открывает переход в «ожидание».

# 2 Гибридный автомат мобильного робота

*Соберём все компоненты на одном рисунке: автомат мобильного робота с тремя режимами и порогами по расстоянию.*




**Движение к цели: $\dot{x} = k \cdot (x_ц - x)$, где $x_ц$ — цель, $k$ — коэффициент; инвариант $d > d_{выкл}$**

**Обход препятствия: робот едет вдоль препятствия со скоростью $v$; инвариант $d_{стоп} < d < d_{выкл}$**

**Остановка: $\dot{x} = 0$, робот стоит; инвариант $d \le d_{стоп}$**

У каждого режима своя непрерывная динамика и свой инвариант по расстоянию $d$ до препятствия. Переходы срабатывают по охранным условиям, а когда препятствие устранено, срабатывает сброс состояния.

# 2 Как развивается траектория гибридного автомата

*Проследим по шагам, как во времени развивается траектория: она то течёт непрерывно, то прыгает.*

**1 Течение**

В режиме q состояние меняется по закону $\dot{x} = f_q(x)$, пока остаётся внутри инварианта Inv(q).

**2 Проверка охраны**

На каждом такте контроллер проверяет, не попало ли состояние в охранное условие G(e) какого-либо перехода.

**3 Скачок**

Если условие выполнено, активный режим меняется мгновенно: на скачок не уходит ни доли секунды.

**4 Сброс**

Состояние пересчитывается по правилу $x^+ = R(e, x)$, например скорость меняется скачком при ударе.

**5 Новое течение**

Состояние снова меняется непрерывно, но уже по закону нового режима и в его инварианте.

Гибридное время двумерно: его описывают парой (t, j), где t — обычное время, а j — число совершённых скачков.

# 2 Подводные камни гибридных автоматов

*У гибридных автоматов есть свойства, которых нет ни у дифференциальных уравнений, ни у обычных конечных автоматов.*

– Зеноновское поведение — бесконечно много переключений за конечное время; так ведёт себя модель мяча с затухающими отскоками.

– Против него в контроллер вводят гистерезис (разные пороги входа и выхода) и минимальное время пребывания в режиме.

– Известный результат: в общем случае нет алгоритма, который ответит, достижимо ли из старта опасное состояние автомата.

– Поэтому строгая проверка возможна лишь для узких классов (временны́е, прямоугольные автоматы) или приближённо.

– Моделируют такие системы в Stateflow и Ptolemy II, а достижимость приближённо проверяют в SpaceEx и Flow*.

**Ключевая идея**

Устойчивость гибридной системы не сводится к устойчивости отдельных режимов: закон переключения — полноправная часть модели, а не техническая деталь.

# ? Как думаете?

Складской робот у края зоны препятствия переключается между режимами «еду к цели» и «объезжаю» 50 раз в секунду: вход в «объезд» и выход из него заданы одним и тем же порогом расстояния. Что исправит ситуацию корректнее всего?

- **A** Увеличить частоту опроса дальномеров
- **B** Разнести пороги входа и выхода (d_вкл < d_выкл) и задать минимальное время в режиме
- **C** Сгладить сигнал дальномера фильтром низких частот
- **D** Уменьшить коэффициент усиления регулятора в обоих режимах

# ? Как думаете?

Складской робот у края зоны препятствия переключается между режимами «еду к цели» и «объезжаю» 50 раз в секунду: вход в «объезд» и выход из него заданы одним и тем же порогом расстояния. Что исправит ситуацию корректнее всего?

**A** Увеличить частоту опроса дальномеров

**B** Разнести пороги входа и выхода (d_вкл < d_выкл) и задать минимальное время в режиме

**C** Сгладить сигнал дальномера фильтром низких частот

**D** Уменьшить коэффициент усиления регулятора в обоих режимах

**ОТВЕТ**

Вариант 2: гистерезис и минимальное время пребывания устраняют саму причину — общий порог входа и выхода, а не маскируют симптом.

# 3 Что гибридный подход даёт группе роботов

*Мы увидели гибридность одного робота; теперь посмотрим, что она даёт группе и чем за неё приходится платить.*

– Комплексность: задача группы тянется от распределения заданий до стабилизации приводов, и ни один метод её целиком не покрывает.

– Адаптивность: обучаемые модули подстраиваются под незнакомую среду, а аналитические сохраняют гарантии в обычных условиях.

– Безопасность: простой проверяемый супервизор страхует «умный» модуль и перехватывает управление у опасной границы.

– Эффективность: тяжёлые расчёты уходят на сервер, лёгкие остаются на борту, а обмен по событиям экономит радиоканал.

– Плата: проектирование, анализ и отладка усложняются, и появляются непредвиденные взаимодействия между модулями.

**Ключевая идея**

Подход runtime assurance («гарантия во время работы»): интеллектуальный компонент оптимизирует поведение, а простой проверяемый супервизор перехватывает управление у опасной границы. Так делают в авиации и автономном транспорте.

# 3 Схема 1: центр планирует, роботы исполняют сами

*Первое измерение гибридности — как поделить решения между центром и роботами. Начнём с самой распространённой схемы.*

– Центр (сервер, наземная станция или ведущий робот) видит всю обстановку и ищет планы $p_1$, …, p_N, минимизирующие общий критерий J.

– Каждый робот i сам замыкает быстрый контур слежения u_i = k_i(x_i, p_i): по своему состоянию x_i отрабатывает план p_i.

– Достоинства схемы: план оптимален для всей группы, а бортовые вычислители роботов разгружены.

– Недостатки: всё зависит от центра и канала связи, а план медленно реагирует на внезапные изменения.

– Мелкие препятствия и возмущения робот парирует сам, не обращаясь к центру.

**Кейс**

Флот складских роботов: сервер назначает задания и резервирует коридоры движения, а роботы сами отрабатывают траектории и уклоняются от локальных препятствий.

# 3 Схемы 2 и 3: надзор центра и переменная централизация

*Если центр не планирует, а лишь надзирает, или если роль центра меняется на ходу, получаются ещё две схемы.*

### Схема 2: роботы планируют, центр надзирает

- Роботы сами строят планы по локальным данным и договариваются, например через аукцион заданий.
- Центр следит за группой и разрешает конфликты, которые роботы не решили между собой.
- Центр может ввести ограничения или остановить робота при угрозе безопасности.
- Пример: управление воздушным движением беспилотников, где диспетчер утверждает маршруты.

### Схема 3: степень централизации меняется

- Степень централизации здесь — переменная величина, а не раз и навсегда принятое решение.
- При хорошей связи и малой группе система работает централизованно, как в схеме 1.
- При потере связи или росте задержек роботы переходят на заранее оговорённые автономные правила.
- Само переключение схем — дискретное событие, поэтому вся группа снова оказывается гибридным автоматом.

# 3 Сравнение архитектур координации

*Сведём три подхода в одну таблицу, чтобы выбор архитектуры стал осознанным проектным решением.*

| Критерий | Централизованная | Децентрализованная | Гибридная |
|---|---|---|---|
| Оптимальность решений | Глобальная | Локальная | Близкая к глобальной |
| Масштабируемость | Низкая | Высокая | Средняя или высокая |
| Устойчивость к отказу узла | Низкая: центр — критическая точка | Высокая | Высокая при резервировании центра |
| Требования к связи | Высокие: каждый робот связан с центром | Локальные: только с соседями | Умеренные, разнесены по уровням |
| Скорость реакции | Ограничена каналом связи | Высокая | Высокая на нижних уровнях |
| Сложность анализа и проверки | Умеренная | Высокая | Высокая |

*Таблица 13.1 пособия. Гибридная архитектура берёт лучшее из двух крайностей, но платит сложностью анализа.*

# 4 Трёхуровневая иерархическая архитектура

*Самая зрелая форма сочетания центра и роя — иерархия из трёх уровней, которые различаются темпом и горизонтом.*

- Стратегический уровень распределяет задачи миссии и ресурсы группы; чаще всего он централизован.
- Тактический уровень строит согласованные траектории, держит строй и разрешает парные конфликты; обычно он децентрализован.
- Исполнительный уровень отрабатывает траекторию и управляет приводами; он полностью локален для каждого робота.
- Сверху вниз идут цели, планы и ограничения, а снизу вверх — состояния, оценки выполнимости и события.
- Событие снизу, например «проезд к стеллажу заблокирован», запускает дискретное перепланирование наверху.

**Ключевая идея**

Уровни различаются не важностью, а горизонтом планирования, темпом работы и степенью абстракции модели: наверху — символические графы задач, внизу — дифференциальные уравнения.

# 4 Что где работает: методы по уровням

*Каждому уровню иерархии соответствует свой набор моделей и методов — посмотрим, какие именно.*

| Уровень | Горизонт и темп | Модели и методы |
|---|---|---|
| Стратегический | Минуты и часы, доли герца | Графы задач, сети Петри, задачи о назначениях, MILP |
| Тактический | Секунды, от единиц до десятков герц | Консенсус, потенциальные поля, MPC, поведенческие автоматы |
| Исполнительный | Миллисекунды, сотни герц и выше | ПИД, LQR, бэкстеппинг, скользящие режимы |

*MILP — целочисленное линейное программирование; MPC — управление с прогнозирующей моделью; ПИД — пропорционально-интегрально-дифференциальный регулятор; LQR — линейно-квадратичный регулятор. Чем ниже уровень, тем быстрее, проще и проверяемее алгоритм.*

# 4 Разделение темпов (time-scale separation)

*Разница темпов между уровнями — не случайность, а условие устойчивости всей иерархии.*

**0,1–1 Гц**

Стратегический уровень: обмен с центром и перепланирование

**~10 Гц**

Тактический уровень: супервизор и уставки скорости v и ω

**~100 Гц**

Исполнительный уровень: ПИД-регулятор с нечёткой подстройкой

**×10**

Типичный разрыв темпов между соседними уровнями

- Верхний уровень видит нижний как безынерционного исполнителя, а нижний видит команды верхнего как медленные уставки.
- Теория сингулярных возмущений: если нижний контур сходится намного быстрее смены команд, из устойчивости уровней следует устойчивость иерархии.
- Если разрыв нарушить, например ускорить перепланирование до темпа слежения, появляются колебания.

# 5 Комбинации методов: рой с обучением, игры с консенсусом

*Второе измерение гибридности — сочетание разнородных алгоритмов. Разберём две показательные пары.*

## Роевые правила и обучение с подкреплением

- Обучение с подкреплением (RL) настраивает не моторы, а веса правил Рейнольдса и радиусы взаимодействия.
- Пространство действий от этого резко сокращается, а политика наследует масштабируемость роя.
- В схеме CTDE (обучение централизованное, исполнение нет) критик видит всю группу лишь при обучении.
- Однотипные роботы обучают одну общую политику (parameter sharing) и обмениваются опытом.

## Теория игр и консенсус

- Дискретный выбор «кто какую задачу берёт» решает игра: поиск равновесия Нэша, аукцион или коалиции.
- Принятое решение исполняется непрерывно, законами консенсуса и удержания строя.
- Консенсус служит и транспортом: роботы согласуют цены и загрузку локальным усреднением.
- Пример со склада: аукцион раздаёт роботам стеллажи, а консенсус выравнивает скорости в колонне.

# 5 Четыре схемы комбинирования и нейросимвольные системы

*Почти любую сборную систему управления можно разложить на четыре типовые схемы соединения методов.*

– Последовательная схема: один метод вырабатывает уставку для другого, например планировщик задаёт траекторию регулятору.

– Параллельная схема с арбитражем: несколько методов предлагают команды, а супервизор выбирает одну или смешивает их.

– Иерархическая схема разносит методы по уровням, а встроенная позволяет одному методу настраивать параметры другого.

– В нейросимвольной системе нейросеть распознаёт сцену, а планировщик на языках STRIPS или PDDL строит план действий.

– Открытые вопросы: как передать неуверенность нейросети в строгую логику планировщика и как объяснить поведение.

**Пример**

Сервисный робот: нейросеть строит символическое описание сцены («дверь закрыта»), планировщик строит план, автомат режимов управляет исполнением, а обучение с подкреплением отвечает за локальную навигацию.

# Конец первой части

*Мы разобрали язык гибридных систем и архитектуры координации. После перерыва — нечёткая логика, устойчивость при переключениях, событийный обмен и барьерные функции безопасности.*

# Часть 2. Нечёткая логика, переключения, события и безопасность

*В первой части мы описали гибридную систему как автомат с режимами и разложили управление группой по уровням; теперь разберём конкретные механизмы — нечёткие регуляторы, гарантии при переключениях, событийный обмен и фильтр безопасности.*

# 6 Нечёткая логика: зачем она в робототехнике

*Первый механизм превращает словесные правила эксперта в закон управления — это нечёткая логика.*

– Нечёткую логику предложил Л. Заде в 1965 году; нечёткое множество A задаётся функцией принадлежности μ_A со значениями от 0 до 1.

– Значение μ_A(x) — степень, с которой x принадлежит A: расстояние 0,8 м «близко» со степенью 0,6 и «средне» со степенью 0,4.

– Функции принадлежности обычно берут треугольными, трапецеидальными или гауссовыми.

– Регулятор работает без точной модели объекта, например когда уравнение контакта колеса со скользким полом неизвестно.

– Базу правил можно прочитать и обсудить с экспертом, в отличие от весов нейросети.

**Ключевая идея**

Нечёткий регулятор формализует качественные знания эксперта («далеко», «слегка повернуть») и превращает их в работоспособный закон управления без построения модели объекта.

# 6 Цикл нечёткого вывода по Мамдани

*Посмотрим, как нечёткий регулятор за один такт превращает измерение в команду; это пять шагов вывода по Мамдани.*

**1 Фаззификация**

Измеренное d = 0,8 м переводится в степени принадлежности термам «близко», «средне», «далеко»: 0,6; 0,4; 0.

**2 Активация правил**

Степень условия каждого правила считается t-нормой — минимумом или произведением степеней его частей.

**3 Импликация**

Заключение каждого правила усекается или масштабируется степенью его условия.

**4 Агрегация**

Усечённые заключения всех сработавших правил объединяются операцией максимума в одно нечёткое множество.

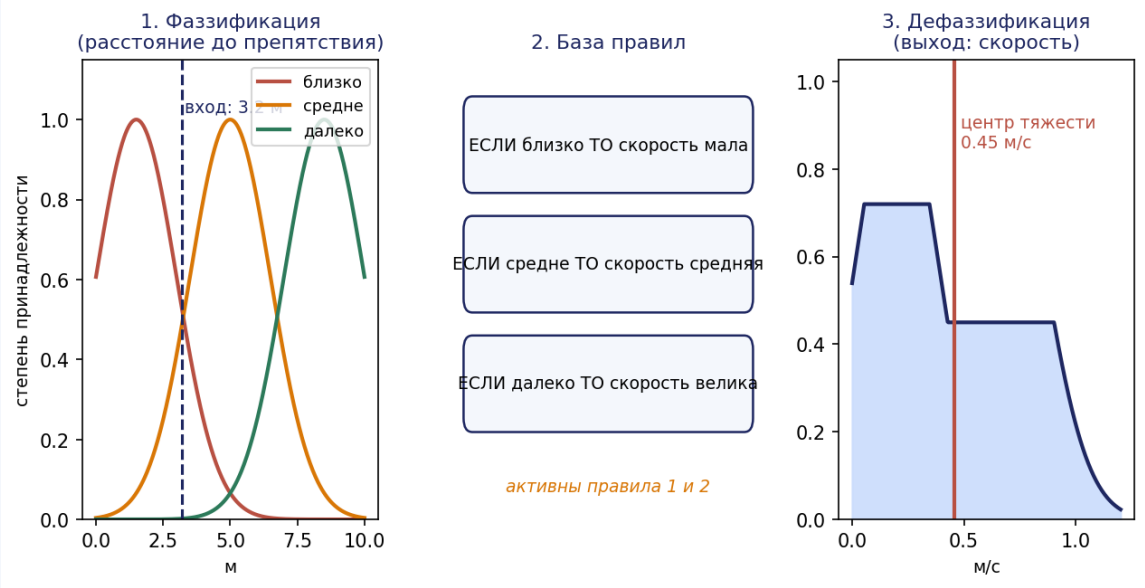
**5 Дефаззификация**

Метод центра тяжести превращает это множество в одно чёткое значение управления u*.

Цикл состоит только из арифметики без итераций, поэтому его повторяют на каждом такте управления.

# 6 Три этапа нечёткого вывода

*Те же этапы на рисунке: видно, как перекрытие термов делает выход регулятора плавным.*




**Фаззификация: расстояние d переводится в степени принадлежности термам**

**База правил: «если d есть A, то скорость v есть B»**

**Дефаззификация: центр тяжести даёт чёткую скорость v**

Слева вход «расстояние до препятствия» разложен по термам «близко», «средне», «далеко»; в центре — база правил; справа центр тяжести итогового множества даёт чёткую скорость.

# 6 Дефаззификация

*Последний шаг вывода превращает нечёткое множество в число; запишем его для двух типов регуляторов.*

$$u^* = \frac{\int u\,\mu(u)\,du}{\int \mu(u)\,du} \qquad\qquad u^* = \frac{\sum_k w_k\,u_k}{\sum_k w_k}$$

- Здесь u — возможное значение управления, μ(u) — итоговая функция принадлежности выхода после агрегации, u* — чёткое управление.
- Левая формула (Мамдани) находит центр тяжести фигуры под μ(u); правая (Такаги — Сугено) — среднее заключений $u_k$ с весами $w_k$.
- Здесь $w_k$ — степень активации k-го правила, $u_k$ — его чёткое заключение: чем сильнее сработало правило, тем сильнее оно тянет выход.
- Пример: правило «скорость 0,2» сработало со степенью 0,4, правило «скорость 0,5» — со степенью 0,3; тогда u* = 0,23 / 0,7 ≈ 0,33.

Центр тяжести гарантирует непрерывную зависимость выхода от входов

# 6 Регуляторы Мамдани и Такаги — Сугено

*Два типа регуляторов различаются видом заключения правила, и от этого зависит, когда какой выбирать.*

### Мамдани

- Заключение правила — нечёткое множество, например «скорость малая».
- Выход получают агрегацией максимумом и затем центром тяжести.
- Правила максимально наглядны для эксперта и заказчика.
- Вычисления дороже, потому что нужно брать интегралы.

### Такаги — Сугено

- Заключение правила — функция входов, в простейшем случае константа.
- Выход — взвешенное среднее заключений, без интегралов.
- Регулятор дешевле в вычислениях и удобнее для анализа.
- Устойчивость можно доказать через линейные матричные неравенства (ЛМН).

# 7 Пример: нечёткий регулятор уклонения складского робота

*Соберём регулятор уклонения для робота с дифференциальным приводом — того, что возит стеллажи по складу.*

- Входы: d_L, d_F, d_R — минимальные расстояния до препятствий в левом, переднем и правом секторах дальномера.
- Каждый вход описан тремя термами — Б (близко), С (средне), Д (далеко) — с трапециями и треугольниками.
- Выходы: линейная скорость v (от «стоп» до «большая») и угловая скорость ω (от «резко влево» до «резко вправо»).
- Стратегия эксперта: впереди свободно — ехать быстро; впереди близко — тормозить и отворачивать туда, где свободнее.
- Проклятие размерности: при n входах по m термов полная база содержит $m^n$ правил, здесь $3^3 = 27$.

**Пример**

Для переднего сектора: «близко» — трапеция с ядром [0; 0,4] м и спадом до 0,9 м; «средне» — треугольник с вершиной в 1,0 м; «далеко» — подъём от 1,2 м и ядро от 1,8 м.

# 7 Фрагмент базы правил (таблица 13.2 пособия)

*Стратегию эксперта записывают таблицей правил; каждая строка читается как «если …, то …».*

| № | Слева / впереди / справа | Скорость v | Поворот ω |
|---|---|---|---|
| 1 | Далеко / далеко / далеко | Большая | Прямо |
| 2 | Далеко / средне / далеко | Средняя | Прямо |
| 5 | Далеко / близко / далеко | Малая | Резко влево |
| 6 | Близко / близко / далеко | Малая | Резко вправо |
| 7 | Далеко / близко / близко | Малая | Резко влево |
| 9 | Близко / близко / близко | Стоп | Резко влево, разворот |
| 10 | Средне / средне / средне | Средняя | Прямо |

*Правило 5 читается так: «если слева далеко, впереди близко и справа далеко, то ехать медленно и резко поворачивать влево». В полной базе пособия 10 правил; отворот всегда в сторону большего свободного пространства.*

# 7 Числовой разбор одного такта

*Прогоним регулятор на конкретных числах, чтобы увидеть, откуда берётся плавность управления.*

– Измерено: слева 2,0 м (далеко 1,0), впереди 0,7 м (близко 0,4, средне 0,3), справа 1,6 м (средне 0,2, далеко 0,67).

– Правило 5 «далеко, близко, далеко» срабатывает со степенью min(1,0; 0,4; 0,67) = 0,4: малая скорость, резко влево.

– Правило 2 «далеко, средне, далеко» срабатывает со степенью min(1,0; 0,3; 0,67) = 0,3: средняя скорость, прямо.

– После агрегации и центра тяжести скорость получается между малой и средней, порядка 0,3–0,4 от максимальной.

– Поворот влево выходит умеренным, а не резким, потому что ситуация пограничная между двумя правилами.

**Ключевая идея**

В пограничной ситуации регулятор выдаёт плавный промежуточный отклик — интерполяцию между двумя экспертными правилами. Жёсткий автомат выбрал бы ровно одно поведение и дал бы скачок управления.

# ? Проверим понимание

Складской робот объезжает препятствие. Почему нечёткий регулятор уклонения даёт более гладкую траекторию, чем пороговый алгоритм «если впереди ближе $d_0$ — повернуть с постоянной угловой скоростью $\omega$»?

**A** Потому что нечёткий регулятор использует больше датчиков

**B** Потому что перекрытие термов даёт интерполяцию между правилами

**C** Потому что нечёткий регулятор обучается на данных во время движения

**D** Потому что нечёткий регулятор работает на более высокой частоте

# Проверим понимание

Складской робот объезжает препятствие. Почему нечёткий регулятор уклонения даёт более гладкую траекторию, чем пороговый алгоритм «если впереди ближе $d_0$ — повернуть с постоянной угловой скоростью $\omega$»?

**A** Потому что нечёткий регулятор использует больше датчиков

**B** Потому что перекрытие термов даёт интерполяцию между правилами

**C** Потому что нечёткий регулятор обучается на данных во время движения

**D** Потому что нечёткий регулятор работает на более высокой частоте

**ОТВЕТ**

Вариант 2: перекрывающиеся функции принадлежности заставляют несколько правил срабатывать одновременно, и выход становится взвешенным компромиссом.

# 7 Нечёткая подстройка ПИД-регулятора

*Нечёткий блок может не управлять сам, а подстраивать классический регулятор — это встроенная схема комбинирования.*

- ПИД-регулятор (пропорционально-интегрально-дифференциальный) u = K_p·e + K_i·∫e dτ + K_d·ė хорош лишь вблизи одной рабочей точки.
- Пример со склада: робот с полным стеллажом намного тяжелее пустого, и одни и те же коэффициенты K_p, K_i, K_d работают хуже.
- При нечёткой подстройке (fuzzy gain scheduling) нечёткий блок следит за ошибкой e и её скоростью ė и плавно меняет коэффициенты.
- Типичное правило подстройки: «если ошибка большая и растёт, то увеличить K_p и уменьшить K_d».
- Структура и надёжность ПИД-контура сохраняются, а адаптивность добавляется без точной модели объекта.

# 8 Нейро-нечёткие системы: ANFIS

*Главная слабость нечёткого регулятора — ручная настройка; нейро-нечёткие сети подбирают параметры по данным.*

– ANFIS (Adaptive Neuro-Fuzzy Inference System, адаптивная нейро-нечёткая система вывода) предложил Дж.-Ш. Р. Джанг в 1993 году.

– Сеть из пяти слоёв реализует вывод Такаги — Сугено: принадлежности, силы правил $w_k$, их нормировка, заключения, сумма.

– Обучение гибридное: линейные параметры заключений находят методом наименьших квадратов, остальные — обратным распространением.

– Обученная сеть читается как набор нечётких правил, поэтому её можно показать эксперту, в отличие от «чёрного ящика».

– Применения: обратная динамика приводов, настройка регуляторов уклонения, обучение поведению по демонстрациям оператора.

**Ключевая идея**

ANFIS убирает ручную настройку функций принадлежности, сохраняя читаемость правил. Ограничения: число параметров быстро растёт с числом входов, и нужна представительная обучающая выборка.

# 9 Переключаемые системы: парадокс устойчивости

*Робот с набором регуляторов и супервизором — это переключаемая система. Сохранится ли при переключениях устойчивость?*

– Переключаемая система записывается как $\dot{x} = f_{\sigma(t)}(x)$, где сигнал $\sigma(t)$ кусочно-постоянен и указывает номер активного режима.

– Устойчивость означает, что малые отклонения не растут и затухают; например, сбитый робот сам возвращается на маршрут.

– Известный результат: есть две устойчивые линейные системы $\dot{x} = A_1 x$ и $\dot{x} = A_2 x$, переключения между которыми уводят траекторию в бесконечность.

– Верно и обратное: удачное переключение между неустойчивыми режимами способно сделать систему устойчивой.

– Значит, устойчивость каждого режима по отдельности ещё не гарантирует устойчивости системы с переключениями.

**Почему так происходит**

Каждый режим по-своему «закручивает» траекторию. Если переключаться в неудачные моменты, рост нормы состояния накапливается от витка к витку, и траектория уходит в бесконечность.

# 9 Устойчивость и функция Ляпунова простыми словами

*Чтобы получить гарантии при переключениях, вспомним инструмент четвёртой лекции — функцию Ляпунова.*

## Функция Ляпунова V(x)

- Это скалярная функция, аналог энергии отклонения: она не меньше нуля и равна нулю только в равновесии.
- Если V убывает вдоль любой траектории системы, отклонение «тратит энергию» и затухает.
- Тогда система устойчива, и для этого вывода не нужно решать дифференциальные уравнения.
- Пример со склада: V — квадрат отклонения робота от маршрута; он уменьшается с каждым тактом слежения.

## Что меняется при переключениях

- У каждого режима может быть своя функция Ляпунова, и каждая убывает, пока работает её режим.
- Но в момент переключения функция нового режима может оказаться больше, то есть «энергия» скачком растёт.
- Если скачки роста сильнее, чем убывание между переключениями, траектория расходится.
- Поэтому нужна либо одна общая функция для всех режимов, либо ограничение частоты переключений.

9

# Теорема об общей функции Ляпунова

*Первый путь к гарантии — найти одну функцию Ляпунова, которая убывает во всех режимах сразу.*

$$\dot{V}(x) = \nabla V(x) \cdot f_q(x) < 0 \quad \forall q, \forall x \neq 0$$

- Здесь V(x) — положительная при x ≠ 0 функция, ∇V — её градиент, f_q(x) — закон движения в режиме q, V̇ — скорость изменения V.
- Теорема: если одна V убывает вдоль траекторий каждого режима при всех x ≠ 0, система устойчива при любом законе переключения.
- Для линейных режимов берут V = $x^T$Px, и условие становится ЛМН (линейными матричными неравенствами) $A_q^T P + P \cdot A_q < 0$.
- Пример: если для режимов «к стеллажу» и «объезд» найдена общая матрица P, супервизор может переключать их как угодно.

Самое сильное, но и самое требовательное условие устойчивости

# 9 Множественные функции Ляпунова и время пребывания

*Общая функция существует не всегда. Тогда устойчивость обеспечивают иначе — ограничивая частоту переключений.*

– Если общей нет, берут свою функцию V_q для каждого режима: при каждом включении режима его V_q должна быть не больше прошлой.

– Теорема о dwell time: если все режимы экспоненциально устойчивы, найдётся τ_d, при паузах не короче которого система устойчива.

– Смысл: пока режим работает, норма состояния успевает упасть сильнее, чем вырастет скачком при переключении.

– Обобщение — среднее время пребывания: редкие быстрые переключения допустимы, если средний интервал достаточно велик.

– Пример со склада: супервизор робота не меняет режим, пока с прошлого переключения не прошло τ_d секунд.

**Ключевая идея**

Инженерные средства — гистерезис на охранных условиях и таймер минимального пребывания в режиме. Они исключают дребезг (chattering) — быстрые переключения на границе условия, которые изнашивают приводы и могут дестабилизировать систему.

# 10 Событийно-управляемое (event-triggered) управление

*Переключаться можно и между «молчу» и «передаю данные». Так устроено событийное управление.*

- Обычная цифровая схема шлёт данные с тактом, рассчитанным на худший случай, и тратит канал и батарею, даже когда ничего не меняется.
- Событийное управление передаёт данные и пересчитывает команду только тогда, когда это действительно нужно.
- Результат П. Табуады: если у исходного закона есть запас устойчивости, то при верном пороге событийная версия тоже устойчива.
- Паузы между событиями не стремятся к нулю; в схеме self-triggered (самозапускающейся) срок следующего обновления считают заранее.
- Пример со склада: робот, стоящий в очереди к стеллажу, почти не шлёт сообщений, а разгоняющийся робот шлёт часто.

**Ключевая идея**

Условия срабатывания событий играют роль охранных условий гибридного автомата: событийное управление — это дискретные переходы, наложенные на непрерывную динамику.

# 10 Условие генерации события

*Момент события задаёт простое пороговое условие на расхождение между отправленным и текущим состоянием.*

$$\|e(t)\| > \sigma \cdot \|x(t)\| + \varepsilon, \quad e(t) = x(t_k) - x(t)$$

– Здесь x(t) — текущее состояние робота, x(t_k) — значение, разосланное соседям в последний раз, e(t) — накопившееся расхождение.

– σ от 0 до 1 — относительный порог, ε — небольшой абсолютный порог, чтобы события не сыпались непрерывно возле нуля.

– Смысл: робот рассылает новое состояние, как только старое значение устарело больше чем на долю σ от текущего плюс ε.

– Пример: при σ = 0,1 и ‖x‖ = 5 м новое сообщение уйдёт, когда расхождение превысит 0,5 м + ε.

В событийном консенсусе трафик сокращается на порядки, а сходимость сохраняется

11

# Барьерные функции безопасности (CBF)

*Устойчивость гарантирует, что робот придёт куда надо. Теперь нужна гарантия, что по дороге он ни во что не врежется.*

– Барьерная функция управления (CBF, Control Barrier Function) h(x) задаёт безопасное множество условием $h(x) \geq 0$.

– Задача не в том, чтобы прийти в это множество, а в том, чтобы никогда его не покинуть.

– Это свойство называют прямой инвариантностью: если система стартовала внутри множества, она останется внутри.

– Для роботов i и j на складе: $h(x) = \|p_i - p_j\|^2 - d_{безоп}^2$, где p — положения, $d_{безоп}$ — допустимое сближение.

– Идея похожа на потенциальные поля из четвёртой лекции, но даёт гарантию безопасности, а не эвристику.

**Ключевая идея**

Функция Ляпунова отвечает за то, чтобы система сходилась куда надо. Барьерная функция отвечает за то, чтобы по дороге она никуда не влезла. Это два разных требования.

11

# Условие инвариантности безопасного множества

*Требование «не покидать безопасное множество» записывается одним неравенством на скорость изменения h.*

$$\dot{h}(x,u) \geq -\alpha(h(x)), \quad h(x) \geq 0$$

- Здесь h(x) — барьерная функция, ḣ(x, u) — скорость её изменения при управлении u, α — возрастающая функция с α(0) = 0, обычно α(h) = γ·h.
- Утверждение: если на каждом шаге выбирать u, удовлетворяющее неравенству, множество h(x) ≥ 0 не будет покинуто никогда.
- Смысл: вдали от границы h может убывать быстро, а у самой границы разрешённая скорость убывания h падает до нуля.
- Для систем вида ẋ = f(x) + g(x)·u неравенство линейно по u; пример: два робота сближаются, но всё медленнее у допуска.

Одно неравенство превращает требование безопасности в ограничение на управление

# 11 Фильтр безопасности на каждом такте

*Неравенство барьера работает как фильтр, который на каждом такте поправляет команду любого регулятора.*

**1 — Номинальная команда**

Её выдаёт любой регулятор: ПИД, MPC (управление с прогнозом), нечёткий или обученный методом RL.

**2 — Сборка ограничений**

Для каждого соседа и каждого препятствия строится своё неравенство $\dot{h} \geq -\alpha(h)$.

**3 — Квадратичная задача**

Решается QP (задача квадратичного программирования): $\min \|u - u_{ном}\|^2$ при барьерных и приводных ограничениях.

**4 — Решение в такте**

Целевая функция квадратична, ограничения линейны, поэтому задачу решают быстро, в пределах такта управления.

**5 — Выдача на приводы**

Пока безопасно, $u^*$ совпадает с $u_{ном}$; у границы команда меняется минимально, ровно насколько требует барьер.

Минимально вмешивающийся фильтр безопасности (minimally invasive safety filter) меняет команду ровно настолько, насколько необходимо.

11

# Два способа обеспечить безопасность

*Теперь у нас два способа обеспечить безопасность — дискретный и непрерывный. Сравним их по свойствам.*

## Переключение режимов супервизором

- Супервизор выбирает регулятор из набора, например включает «объезд» при близком препятствии.
- Границы режимов явные, а от дребезга защищают гистерезис и время пребывания.
- Супервизор легко проверить автоматически: model checking перебирает все его состояния.
- Минусы: управление меняется скачками, и остаётся риск дребезга.

## Непрерывный фильтр на барьерных функциях (CBF)

- Один регулятор работает всё время, а поверх него стоит ограничение задачи QP.
- Граница плавная: функция $\alpha$ задаёт, насколько мягко робот к ней подходит.
- Гарантию, что робот не покинет безопасное множество, доказывают аналитически.
- Минусы: нужна модель динамики и решатель квадратичной задачи на борту.

# 12 Гибридный контроллер робота: один такт Δt = 10 мс

*Соберём лекцию в один бортовой цикл робота транспортной группы: каждый шаг взят с одного из предыдущих слайдов.*

**1 — Оценка и приём**

Робот оценивает состояние $x_i$ по ИНС (инерциальной навигации) и одометрии, читает дальномеры и сообщения.

**2 — Событийная рассылка**

Если $\|x_i - x_{посл}\| > \sigma \cdot \|x_i\| + \varepsilon$, робот рассылает соседям своё состояние и запоминает его.

**3 — Супервизор**

Выбирает режим «следование», «уклонение», «согласование» или «авария» с гистерезисом и лишь после $\tau_d$.

**4 — Уставки режима**

Режим даёт уставки $v$ и $\omega$: слежение за планом, нечёткий регулятор, консенсус строя или (0, 0) при аварии.

**5 — Исполнение**

ПИД-регулятор с нечёткой подстройкой отрабатывает уставки; поверх можно поставить фильтр на барьерных функциях (CBF).

Стратегический уровень работает асинхронно, с темпом 0,1–1 Гц: обменивается с центром и перепланирует маршрут.

# 12 Проверка, испытания и прикладные примеры

*Последний вопрос проектирования: как убедиться, что собранный гибридный контроллер работает правильно.*

– Дискретное ядро — супервизор — проверяют формально программами проверки моделей (model checking) UPPAAL, SPIN и NuSMV.

– Проверяют отсутствие тупиков, достижимость аварийного режима из любого состояния и инварианты безопасности.

– Совместную непрерывно-дискретную динамику анализируют приближённо, вычисляя множества достижимости в SpaceEx и Flow*.

– Массовое моделирование в Gazebo, Webots или Stage проверяет сценарии сбоев, особенно возле охранных условий.

– Натурные испытания идут по нарастающей: один робот, малая группа, вся группа — всегда с независимой аварийной кнопкой.

**Кейс**

Беспилотный автомобиль: автомат дорожных ситуаций, нейросетевое восприятие, MPC (управление с прогнозом) и ПИД на нижнем уровне, а также проверяемый супервизор, ограничивающий команды интеллектуальных модулей.

“

Интеллект — наверх, гарантии — вниз: пусть обучаемые модули предлагают, а простой проверяемый слой решает, что действительно уйдёт на приводы.

*Принцип «гарантии во время работы» (runtime assurance) — от супервизора режимов до фильтра на барьерных функциях*

# Выводы

1. Гибридная система сочетает непрерывное движение и дискретные режимы; её модель — гибридный автомат.
2. Центр и роботы делят решения по трём схемам, а зрелая форма — трёхуровневая иерархия с разделением темпов.
3. Комбинации методов соединяют адаптивность обучаемых модулей с гарантиями аналитических.
4. Нечёткий регулятор даёт плавное управление без модели объекта, а сеть ANFIS настраивает его по данным.
5. Устойчивость режимов не гарантирует устойчивости переключений: нужна общая функция Ляпунова или время пребывания.
6. Событийные схемы передают данные только по необходимости, а фильтр на барьерных функциях охраняет безопасность.
7. Плата за гибрид — сложность, а средства против неё — модульность, проверка ядра, моделирование и испытания.

# Вопросы для самопроверки

1. Чем различаются формальная и инженерная трактовки слова «гибридный» и как они связаны?

2. Из чего состоит гибридный автомат? Что такое зеноновское поведение и как его исключают?

3. Сравните схемы «центр планирует» и «центр надзирает» по оптимальности, живучести и связи.

4. Почему для устойчивости иерархии важен разрыв темпов между уровнями?

5. Как обучение с подкреплением сочетается с роевыми правилами? Почему CTDE — гибридная схема?

6. Опишите этапы нечёткого вывода Мамдани. Чем отличается регулятор Такаги — Сугено?

7. Почему нечёткий регулятор даёт промежуточное управление на границе двух правил?

8. Почему переключение двух устойчивых систем может дать неустойчивость и что такое dwell time?

9. Запишите пороговое условие события. Чем событийный обмен полезен группе роботов?

10. Что гарантирует условие $\dot{h} \geq -\alpha(h)$ и как фильтр CBF решается на каждом такте?